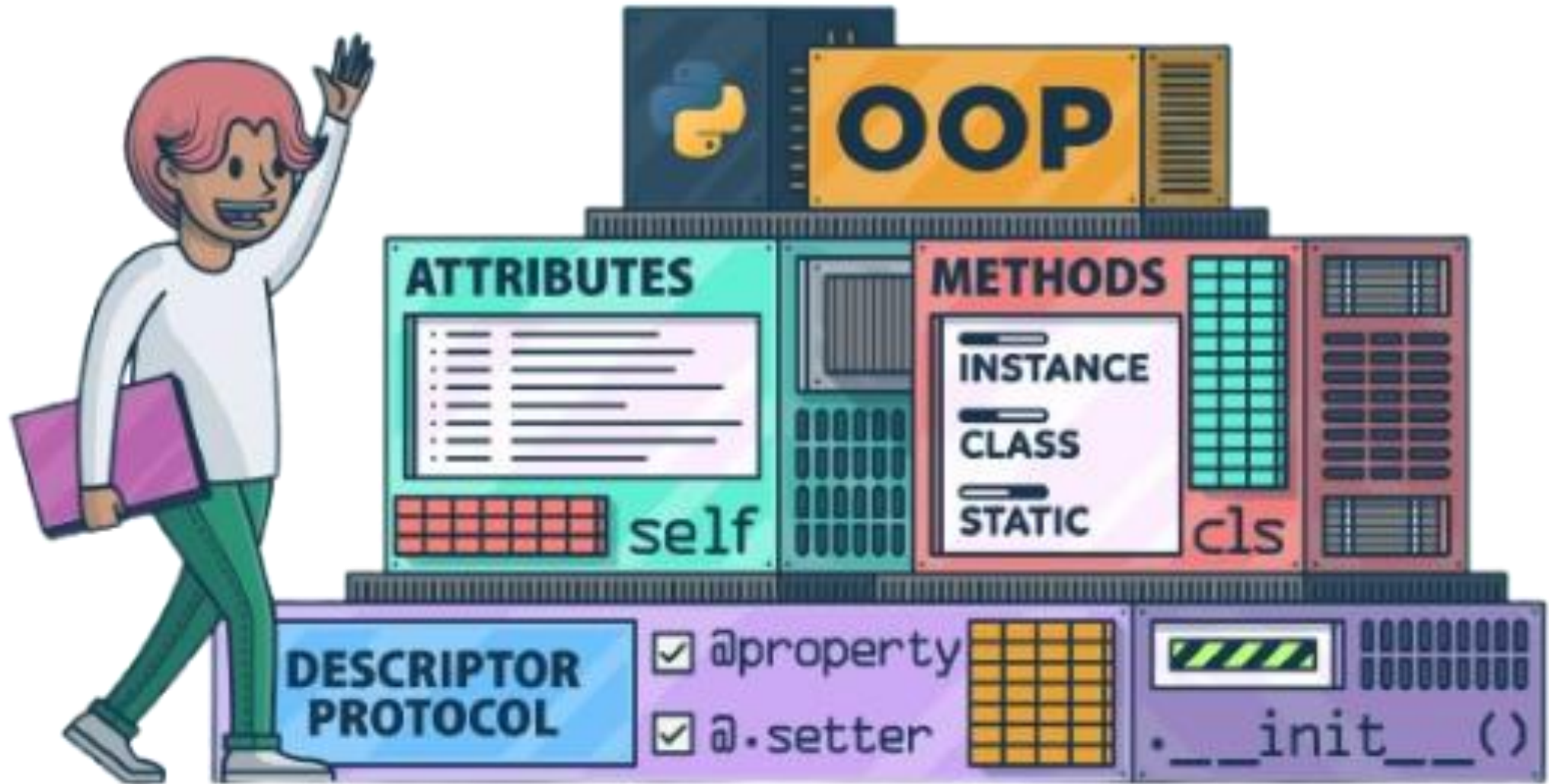
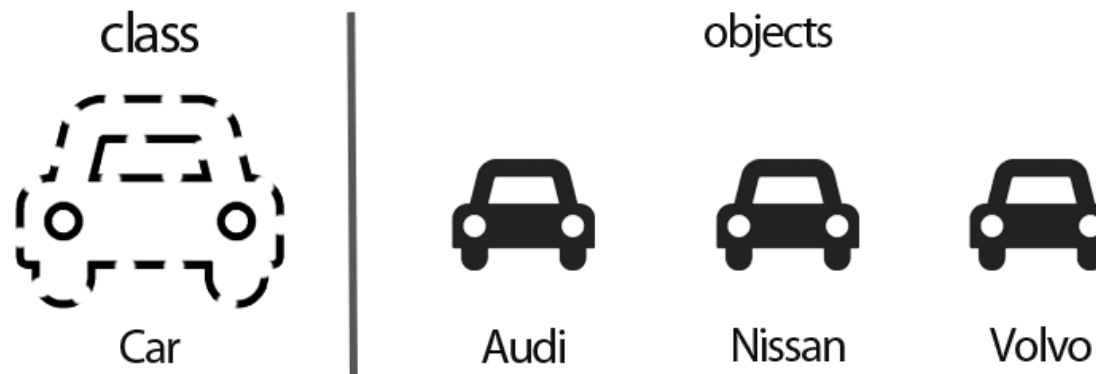




➤ Introduction to OOP in Python

Object-Oriented Programming (OOP) is a core concept in Python that enables developers to create modular, maintainable, and scalable applications.

OOP organizes code by using **classes** and **objects** to model real-world entities and their behaviors. An **object** represents a specific entity with **attributes** (data) and can perform certain actions through **methods**.



OOPs Key Concepts in Python

- **Class**
- **Objects**
- **Encapsulation**
- **Polymorphism**
- **Inheritance**
- **Data Abstraction**



Benefits of OOP in Python

- **Modularity:** Breaks complex code into manageable components.
- **Reusability:** Inheritance allows code reuse and reduces redundancy.
- **Encapsulation:** Protects data through controlled access.
- **Scalability:** Easy to add/modify features without affecting the whole system.
- **Organized Structure:** Enhances readability and collaboration.
- **Maintenance:** Fixes are localized, reducing errors.
- **Reliability:** Reusable components boost robustness.
- **Real-World Modeling:** Mirrors real-life systems for intuitive design.



OOP Limitations

- **Steep Learning Curve:** Difficult for beginners due to complex concepts like classes and inheritance.
- **Higher Complexity:** Large OOP systems can be hard to understand and maintain.
- **Performance Overhead:** Extra abstraction leads to slower execution compared to procedural code.
- **Tight Coupling:** Inheritance can create strong interdependencies between classes.
- **Inheritance Misuse:** Overuse leads to rigid and hard-to-maintain class hierarchies.



➤ What is a Class?

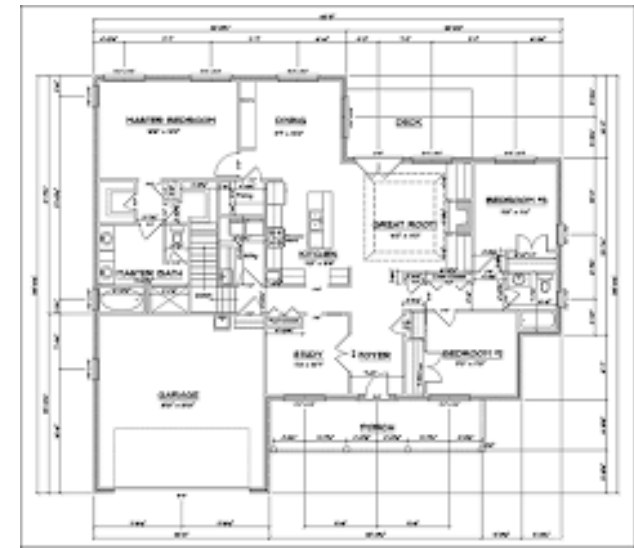
A **class** in Python is a user-defined blueprint for creating objects. It groups related **data (attributes)** and **functions (methods)** into a single unit, making code more organized and reusable. Defining a class introduces a new type, and from this type, we can create multiple **instances** (objects) that share the same structure and behavior.

Classes in Python are created using the **class** keyword.

Attributes are variables defined within a class that represent its **properties or state**.

They can be accessed using the **dot (.) operator**, for example:

```
# define a class
class Dog:
    sound = "bark" # class attribute
```



➤ What is an Object?

An **object** is a specific **instance** of a class. It contains its own data (called **instance variables**) and can access the methods defined in its class. Multiple objects can be created from the same class, each having its own unique set of attributes.

Let's create an object from a Dog class:

```
class Dog:
    sound = "bark"

# Create an object from the class
dog1 = Dog()

# Access the class attribute
print(dog1.sound)
```



In the example below, the sound attribute is a **class attribute**. It is shared across **all instances** of the Dog class and can be accessed using the class name or any instance.



➤ Using `__init__()` Function

In Python, a class includes a special method called `__init__()`, which acts as the **constructor**. It is **automatically called** when a new object is created and is used to **initialize the object's attributes**.

```
class Dog:
    species = "Canine" # Class attribute

    def __init__(self, name, age):
        self.name = name # Instance attribute
        self.age = age # Instance attribute
```

Explanation:

- **class Dog:** Defines a class named Dog.
- **species:** A class attribute shared by all instances of the class.
- **`__init__` method:** Initializes the name and age attributes when a new object is created.



➤ Initiate Object with `__init__()`

```
class Dog:
    species = "Canine" # Class attribute

    def __init__(self, name, age):
        self.name = name # Instance attribute
        self.age = age # Instance attribute

# Creating an object of the Dog class
dog1 = Dog("Buddy", 3)

print(dog1.name) # Output: Buddy
print(dog1.species) # Output: Canine
```

Explanation:

- **dog1 = Dog("Buddy", 3):** Creates an object of the Dog class with name as "Buddy" and age as 3.
- **dog1.name:** Accesses the instance attribute name of the dog1 object.
- **dog1.species:** Accesses the class attribute species of the dog1 object.



➤ Self Parameter

The **self** parameter is a reference to the **current instance** of the class. It is used inside class methods to **access attributes and other methods** of the same object.

It must be the **first parameter** of any instance method, including `__init__()`, though it is passed **automatically** when the method is called.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(f'{self.name} is barking!')

# Creating an instance of Dog
dog1 = Dog("Buddy", 3)
dog1.bark()
```

Explanation:

- Inside the `bark()` method, **self.name** accesses the **specific dog's name** associated with that object.
- When we call **dog1.bark()**, Python automatically passes the object `dog1` as the **self** parameter. This allows the method to access the attributes (like name) of that particular instance.



➤ `__str__()` Method

The `__str__()` method allows you to define a **custom string representation** of an object.

By default, printing an object or using `str()` returns a generic string like:

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f'{self.name} is {self.age} years old.'    # Correct: Returning a string

dog1 = Dog("Buddy", 3)
dog2 = Dog("Charlie", 5)

print(dog1)
print(dog2)
```



Class and Instance Variables in Python

In Python, variables declared within a class can be categorized as either **class variables** or **instance variables**, and understanding the difference is essential for effective object-oriented programming.

Class Variables

Class variables are shared among **all instances** of a class. They are defined **at the class level**, outside of any methods. Every object created from the class will share the same value for a class variable, unless it is specifically overridden within an object.

Instance Variables

Instance variables are **unique to each object** created from a class. They are typically defined inside the **`__init__()`** method or other instance methods. Each object maintains its **own separate copy** of these variables, which allows different instances to hold different data.



➤ Class and Instance Variables Example

```
class Dog:
    # Class variable
    species = "Canine"

    def __init__(self, name, age):
        # Instance variables
        self.name = name
        self.age = age

# Create objects
dog1 = Dog("Buddy", 3)
dog2 = Dog("Charlie", 5)

# Access class and instance variables
print(dog1.species) # (Class variable)
print(dog1.name)    # (Instance variable)
print(dog2.name)    # (Instance variable)
```

```
# Modify instance variables
dog1.name = "Max"
print(dog1.name)    # (Updated instance variable)

# Modify class variable
Dog.species = "Feline"
print(dog1.species) # (Updated class variable)
print(dog2.species)
```

Explanation:

- **Class Variable (species):** Shared by all instances of the class. Changing Dog.species affects all objects, as it's a property of the class itself.
- **Instance Variables (name, age):** Defined in the __init__() method. Unique to each instance (e.g., dog1.name and dog2.name are different).
- **Accessing Variables:** Class variables can be accessed via the class name (Dog.species) or an object (dog1.species). Instance variables are accessed via the object (dog1.name).
- **Updating Variables:** Changing Dog.species affects all instances. Changing dog1.name only affects dog1 and does not impact dog2.



Getter and Setter Methods

Getter and Setter methods in Python are used to provide **controlled access** to an object's attributes. They play a key role in **encapsulation**, allowing you to safely retrieve or update the values of private attributes.

Unlike some other programming languages, Python doesn't require explicit get and set method names. Instead, it achieves this functionality using **property decorators**.

- **Getter:** Retrieves the value of a private attribute.
- **Setter:** Updates or modifies the value of a private attribute.

These decorators ensure that attribute access remains clean and intuitive while maintaining internal data protection.



➤ Getter and Setter Methods Example

```
class Dog:
    def __init__(self, name, age):
        self._name = name # Conventionally private variable
        self._age = age # Conventionally private variable

    @property
    def name(self):
        return self._name # Getter

    @name.setter
    def name(self, value):
        self._name = value # Setter
```

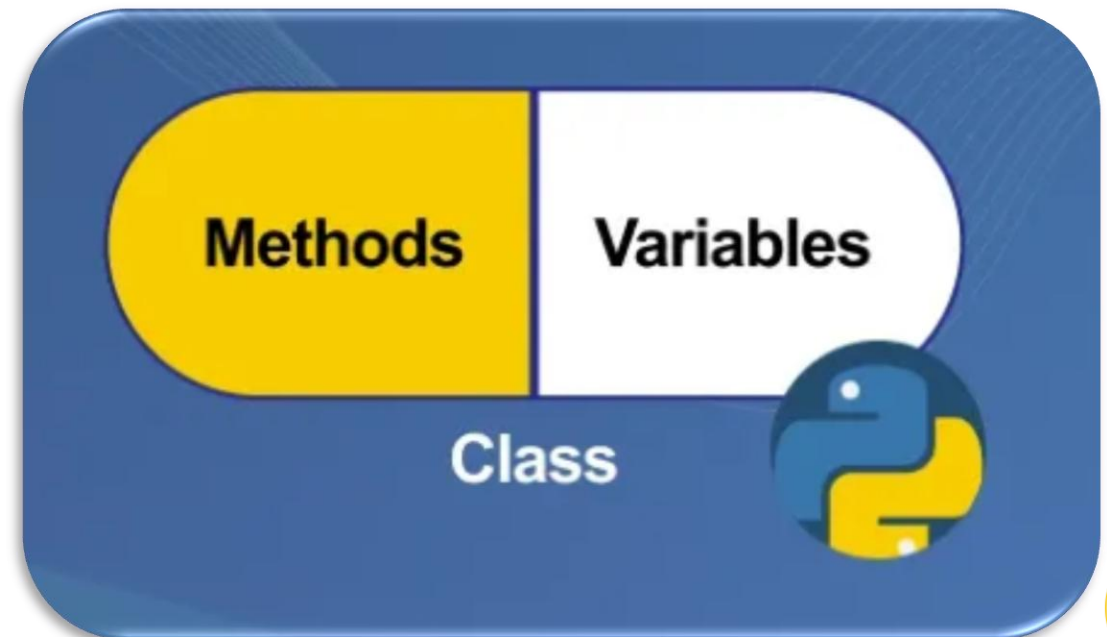
```
@property
def age(self):
    return self._age # Getter

@age.setter
def age(self, value):
    if value < 0:
        print("Age cannot be negative!")
    else:
        self._age = value # Setter
```



➤ Encapsulation in Python

- Encapsulation is the process of bundling data (attributes) and the methods (functions) that operate on that data into a single unit — typically a class.
- It helps restrict **direct access** to internal data and promotes safe data manipulation.
- In Python, encapsulation is achieved using **access specifiers**:



Why Encapsulation?

- Prevents accidental or unauthorized data modification.
- Enforces controlled access using **getter and setter** methods.
- Enhances **modularity** by separating internal logic from external access.
- **Real-world analogy:** Like bank accounts — balance can only be accessed/modified via deposit/withdraw methods.



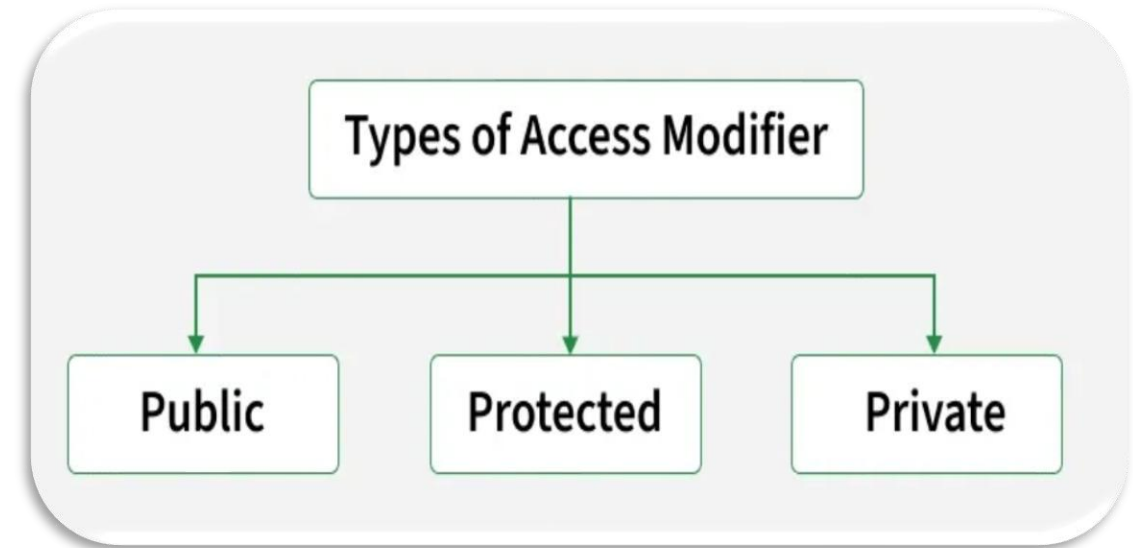
Benefits of Encapsulation

-  Data Protection – Hides critical internal details.
-  Improved Security – Sensitive data is guarded against misuse.
-  Ease of Maintenance – Changes are localized within the class.
-  Code Reusability – Easier to reuse encapsulated classes.
-  Scalability – Supports better organization of large programs.



➤ Access Specifiers in Python

Python supports 3 types of access control:



Specifier	Prefix	Access Level
Public	No underscore	Accessible everywhere
Protected	—	Accessible within class & subclass
Private	—	Only accessible within class (name mangled)



➤ Public Members

- Public attributes and methods can be accessed from **anywhere**.

```
class Public:
    def __init__(self):
        self.name = "John" # Public attribute

    def display_name(self):
        print(self.name) # Public method

obj = Public()
obj.display_name()
print(obj.name)
```

✓ **Explanation:**

- name and display_name() are public.
- Accessible both inside and outside the class.



➤ Protected Members

- Indicated by a single underscore `_`.
- Suggests that the attribute should be used only within the class or its subclasses.

```
class Protected:
    def __init__(self):
        self._age = 30 # Protected attribute

class Subclass(Protected):
    def display_age(self):
        print(self._age)

obj = Subclass()
obj.display_age()
```

 Note: Python doesn't strictly enforce this — **it's a convention.**



➤ Private Members

- Indicated by double underscore `__`.
- Cannot be accessed directly from outside the class.

```
class Private:
    def __init__(self):
        self.__salary = 50000 # Private attribute

    def get_salary(self):
        return self.__salary

obj = Private()
print(obj.get_salary()) # Works
# print(obj.__salary) ❌ Raises AttributeError
```

💡 How it works: Python applies name mangling, renaming `__salary` to `_Private__salary`.



➤ Encapsulation via Getter/Setter

Encapsulation encourages using getters and setters for controlled access:

```
class Employee:
    def __init__(self):
        self.__salary = 0

    def set_salary(self, amount):
        if amount > 0:
            self.__salary = amount
        else:
            print("Invalid salary!")

    def get_salary(self):
        return self.__salary

emp = Employee()
emp.set_salary(50000)
print(emp.get_salary())
```

Why?

- Ensures validation before setting data.
- Prevents invalid or harmful assignments.



➤ Real-World Example - Bank Account

Encapsulation mirrors real-world systems like banking:

```
class BankAccount:
    def __init__(self):
        self.__balance = 0

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Insufficient funds.")

    def get_balance(self):
        return self.__balance
```

```
acc = BankAccount()
acc.deposit(1000)
acc.withdraw(500)
print(acc.get_balance())
```

Explanation:

- Balance is private.
- Access only through methods like deposit, withdraw, get_balance.



➤ Name Mangling in Python

Python internally renames private variables using `_ClassName__varname` format.

```
class Test:
    def __init__(self):
        self.__x = 10

obj = Test()
print(obj._Test__x) # Access via name mangling
```

⚠ **Note:** This allows access but should be avoided — use getter/setter instead.



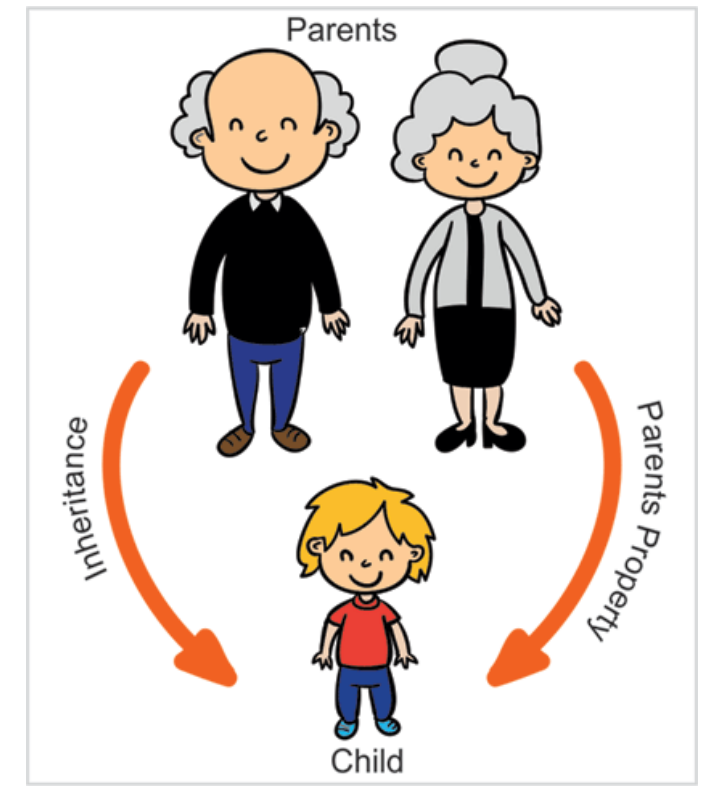
➤ Inheritance in Python

Inheritance is a fundamental concept in **Object-Oriented Programming (OOP)**. It allows a **child class (subclass)** to **inherit** properties and behaviors (methods and variables) from a **parent class (superclass)**.

This enables developers to create new classes that build on existing ones without rewriting code, promoting **code reuse**, **scalability**, and **logical structure**.

Key Concepts:

- The parent class contains common functionality.
- The child class inherits it and can add new features or override existing ones.
- Python supports single, multiple, multilevel, and hierarchical inheritance.



➤ Why Inheritance?

Benefit	Description
✓ Code Reusability	Share common logic across multiple classes
✓ Extensibility	Add new functionality without modifying existing code
✓ Modularity	Organize code into meaningful, reusable components
✓ Hierarchical Design	Reflects real-world relationships, making the code more intuitive

Syntax:

```
class Parent:
    # Parent class code

class Child(Parent):
    # Child class inherits from Parent
```

- **Parent/Base Class:** The class being inherited from
- **Child/Derived Class:** The class that inherits



➤ Simple Example:

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def bark(self):
        print("Dog barks")

d = Dog()
d.speak() # Inherited method
d.bark()  # Own method
```

Explanation:

- Dog inherits the speak() method from Animal
- Dog also defines its own method bark()
- This shows **code reuse** and **extension** of behavior



Types of Inheritance in Python

Python supports the following types:

- 1.Single Inheritance** – One child class inherits from one parent class.
- 2.Multiple Inheritance** – One child class inherits from multiple parent classes.
- 3.Multilevel Inheritance** – A class inherits from a class which is also a subclass.
- 4.Hierarchical Inheritance** – Multiple child classes inherit from a single parent class.
- 5.Hybrid Inheritance** – A mix of two or more types above.



➤ Single Inheritance

Single inheritance allows a **child class** to inherit from **one parent class**.

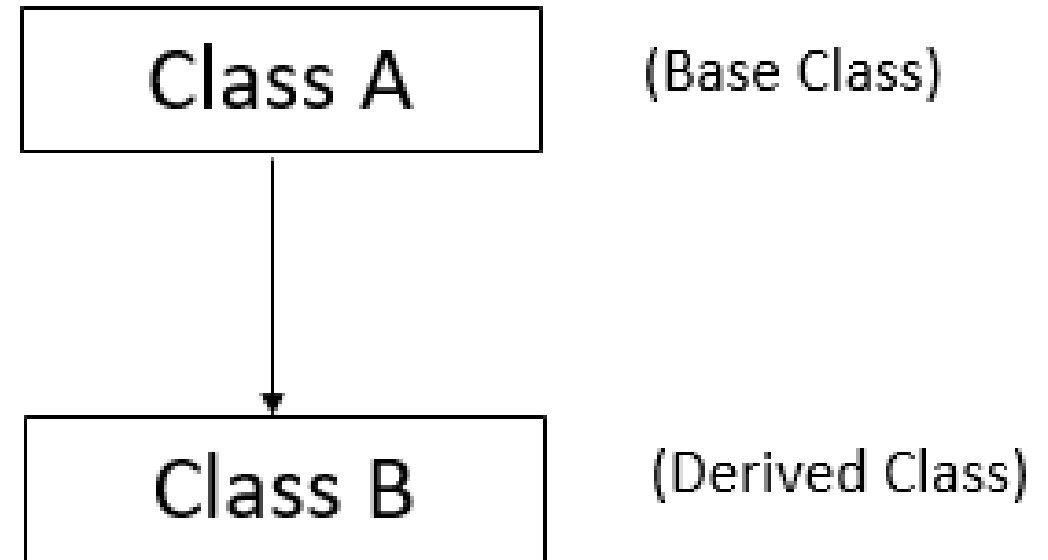
It helps reuse code from the parent while allowing the child to add its own functionality.

Example:

```
class Parent:
    def greet(self):
        print("Hello")

class Child(Parent):
    def welcome(self):
        print("Welcome!")

obj = Child()
obj.greet()    # Inherited from Parent
obj.welcome() # Defined in Child
```



➤ Multiple Inheritance

In multiple inheritance, a **child class inherits from two or more parent classes**.

It allows combining features from multiple sources into one class.

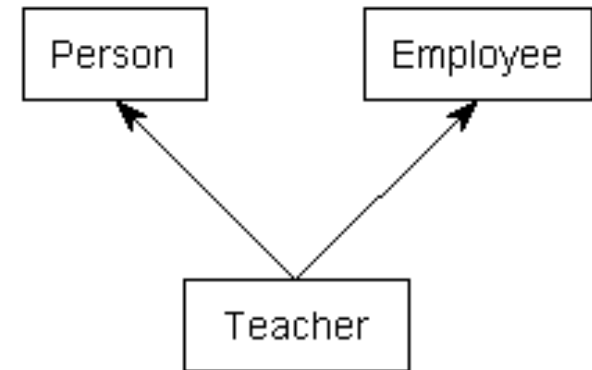
Example:

```
class A:
    def showA(self):
        print("Class A")

class B:
    def showB(self):
        print("Class B")

class C(A, B):
    pass

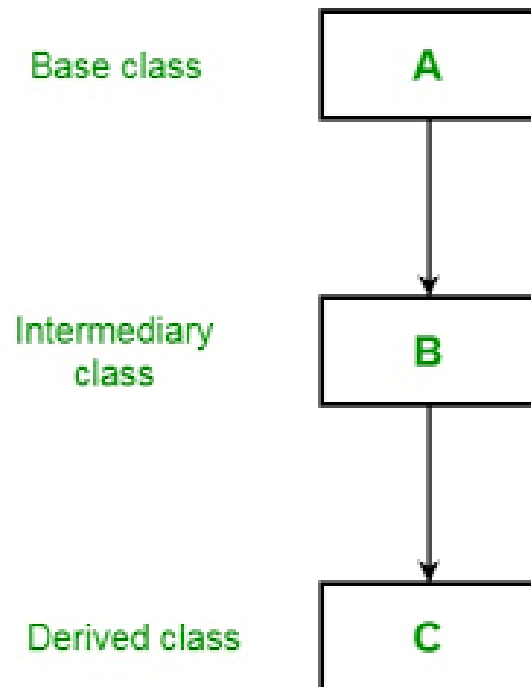
obj = C()
obj.showA() # From class A
obj.showB() # From class B
```



➤ Multilevel Inheritance

In multilevel inheritance, a **class inherits from a child class**, which itself inherits from a **parent class**.

It creates a **chain of inheritance** across multiple levels.



```
class A:
    def funcA(self):
        print("Class A")
```

```
class B(A):
    def funcB(self):
        print("Class B")
```

```
class C(B):
    def funcC(self):
        print("Class C")
```

```
obj = C()
obj.funcA() # Inherited from A
obj.funcB() # Inherited from B
obj.funcC() # Defined in C
```



➤ Hierarchical Inheritance

In hierarchical inheritance, **multiple child classes inherit from a single parent class**.

All child classes share the behavior of the parent but can also have their own specific features.

```
class Parent:
    def show(self):
        print("Parent")

class Child1(Parent):
    pass

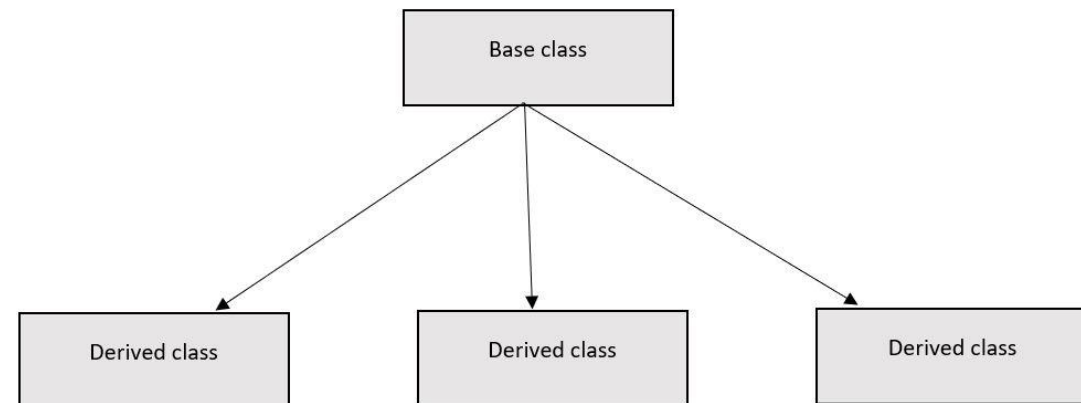
class Child2(Parent):
    pass

obj1 = Child1()
obj2 = Child2()

obj1.show() # Inherited from Parent
obj2.show() # Inherited from Parent
```

Explanation:

- ✓ Child1 and Child2 both inherit from the same Parent
- ✓ They both reuse the show() method from Parent
- ✓ This promotes **code reuse across multiple child classes**



➤ Hybrid Inheritance

Hybrid inheritance is a **combination of two or more types** of inheritance (like single, multiple, multilevel).

It is used when a class structure is complex and requires features from multiple inheritance styles.

```
class A:
    def funcA(self):
        print("Class A")

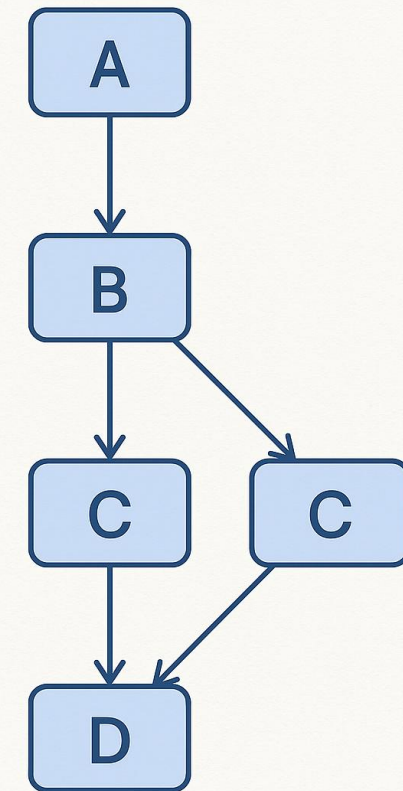
class B(A):
    def funcB(self):
        print("Class B")

class C:
    def funcC(self):
        print("Class C")

class D(B, C):
    def funcD(self):
        print("Class D")
```

obj = D()
 obj.funcA() # From A via B
 obj.funcB() # From B
 obj.funcC() # From C
 obj.funcD() # Defined in D

Hybrid Inheritance



➤ Using super() to Call Parent Method

super() is a built-in function used to call a method from the **parent (super) class** inside a **child class**.

It is commonly used to **extend** or **modify** the behavior of inherited methods without completely overriding them.

Example:

```
class Parent:
    def show(self):
        print("Parent show")

class Child(Parent):
    def show(self):
        super().show() # Calls parent method
        print("Child show")

obj = Child()
obj.show()
```

Output:

```
Parent show
Child show
```



➤ Polymorphism

Polymorphism is a fundamental concept in Object-Oriented Programming (OOP) that allows the same operation to behave differently on different classes.

📌 **Meaning:** “*Poly*” = *many*, “*morph*” = *forms*

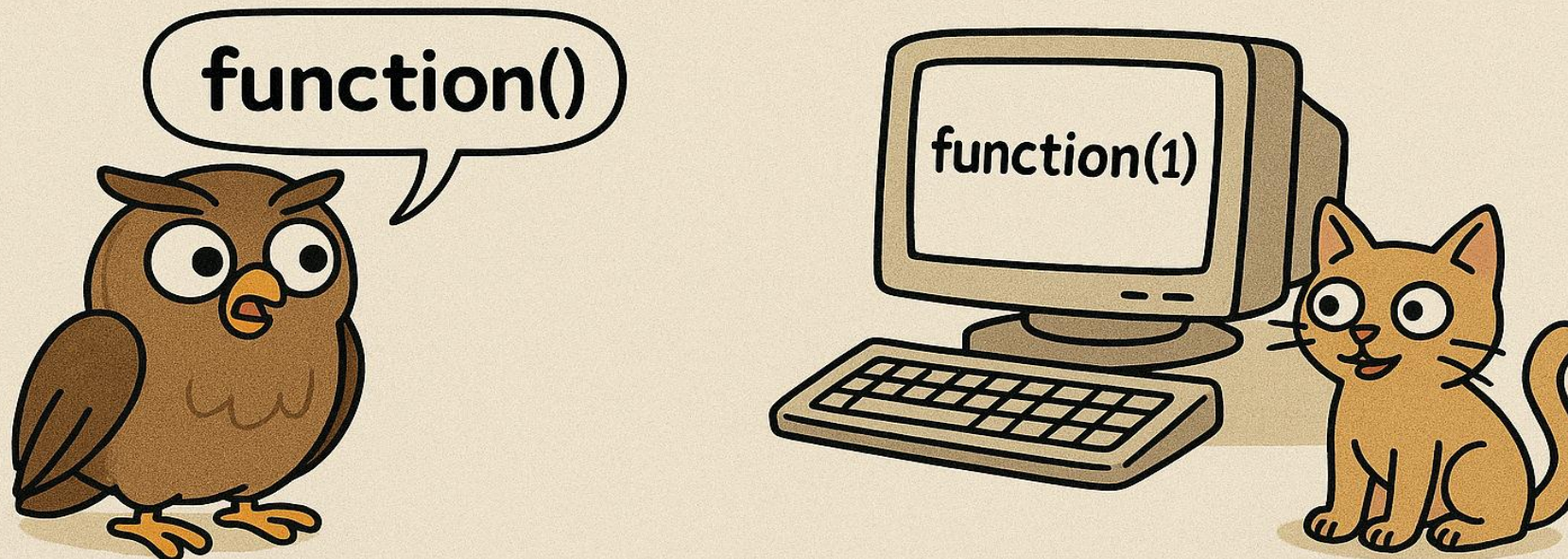
🧠 It enables code reusability, flexibility, and interface consistency.



➤ Types of Polymorphism in Python

Python supports 2 main types of polymorphism:

Type	Description
Compile-time Polymorphism (Static)	Same method name with different parameters. Python simulates this.
Run-time Polymorphism (Dynamic)	Method implementation decided at runtime. Widely supported in Python



➤ Compile-Time Polymorphism (Method Overloading)

Python does **not** support traditional method overloading directly (as in Java/C++), but we can **simulate it** using:

- Default arguments
- Variable-length arguments (*args, **kwargs)

 **Example using default arguments:**

```
class Greet:
    def hello(self, name=None):
        if name:
            print(f'Hello, {name}!')
        else:
            print("Hello!")

g = Greet()
g.hello()      # Hello!
g.hello("Ritesh") # Hello, Ritesh!
```



Another Example of Simulated Overloading

```
class Math:
    def add(self, a=None, b=None, c=None):
        if a and b and c:
            return a + b + c
        elif a and b:
            return a + b
        else:
            return "Invalid input"

m = Math()
print(m.add(2, 3))    # 5
print(m.add(1, 2, 3)) # 6
```

 This mimics method overloading using optional parameters.



➤ Run-Time Polymorphism (Method Overriding)

In run-time polymorphism, the **child class overrides** a method from the parent class to provide a specific implementation.

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def speak(self):
        print("Dog barks")

d = Dog()
d.speak() # Dog barks
```

- ✅ The overridden method is called based on the object type at runtime.



➤ Overriding with super()

The `super()` function is used to call the parent class method from the child class.

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def speak(self):
        super().speak()
        print("Dog barks")

d = Dog()
d.speak()
```

 **Output:**

```
Animal speaks
Dog barks
```



➤ Polymorphism Using Functions and Classes

Different classes having the same method name:

```
class Bird:
    def fly(self):
        print("Bird flies in the sky")

class Airplane:
    def fly(self):
        print("Airplane flies at high altitude")

def start_flying(obj):
    obj.fly()

start_flying(Bird())    # Bird flies in the sky
start_flying(Airplane()) # Airplane flies at high altitude
```

✓ Function accepts any object that has a fly() method — duck typing.



Operator Overloading in Python

Python also supports **operator overloading**, where standard operators behave differently based on operands.

```
class Book:
    def __init__(self, pages):
        self.pages = pages

    def __add__(self, other):
        return self.pages + other.pages

b1 = Book(100)
b2 = Book(200)

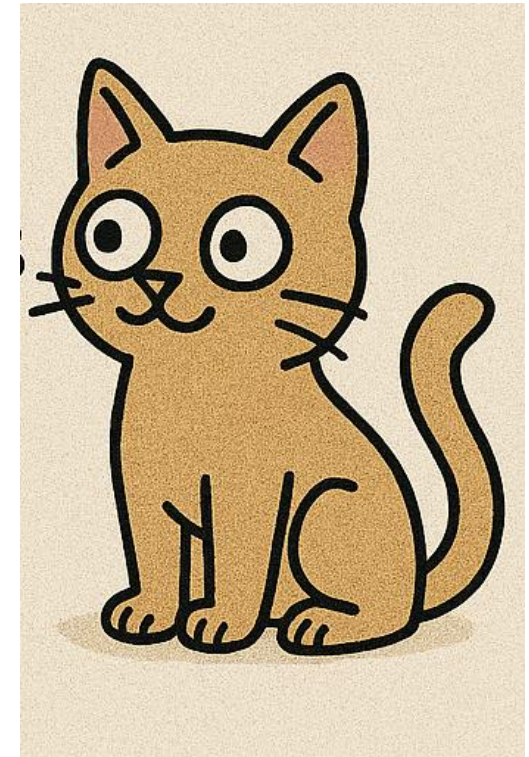
print(b1 + b2) # 300
```

 + operator is overloaded to add Book objects' pages.



➤ Why Polymorphism Matters

- Enhances **code reusability** and **readability**.
- Supports **interface design** (methods stay same, implementations vary).
- Enables **flexibility** and **extensibility** in large codebases.
- Helps implement **design principles** like Liskov Substitution Principle.



➤ What is Abstraction?

Abstraction is the process of **hiding the implementation details** and showing **only essential features** to the user.

🧠 It helps in managing complexity by letting the user interact with an object at a high level, without knowing the internal details.

📁 Real-life Analogy:

- Driving a car — You use the **steering, brake, accelerator**, but don't know how the **engine works** inside.



Real Life Example of Abstraction



➤ Abstraction in OOP

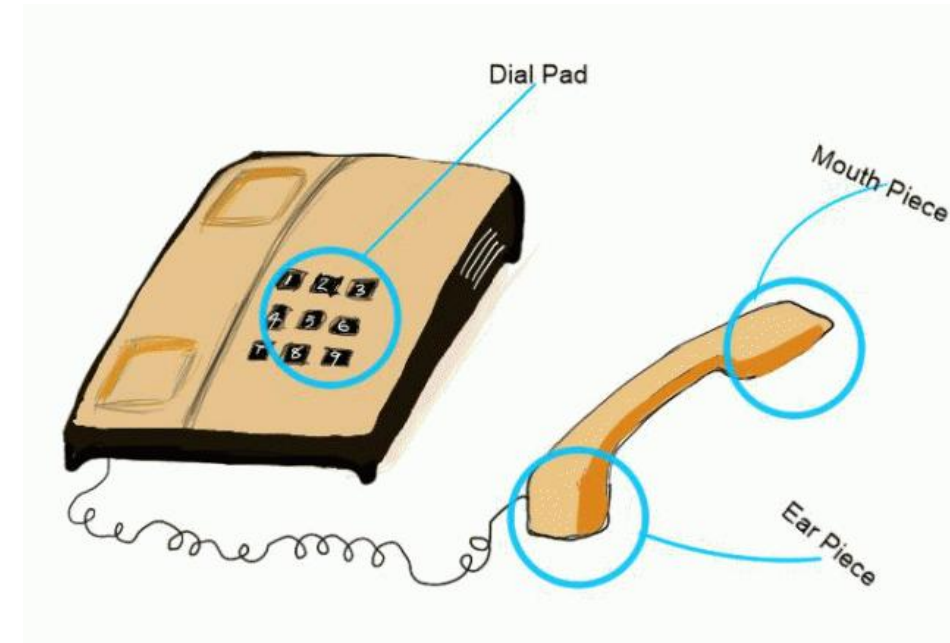
In Object-Oriented Programming, **abstraction** is achieved using:

- **Abstract Classes**
- **Interfaces (via Abstract Base Classes in Python)**



Purpose:

- Define **blueprints** for classes.
- Force child classes to **implement specific methods**.
- Enforce **rules or contracts** across class hierarchies.

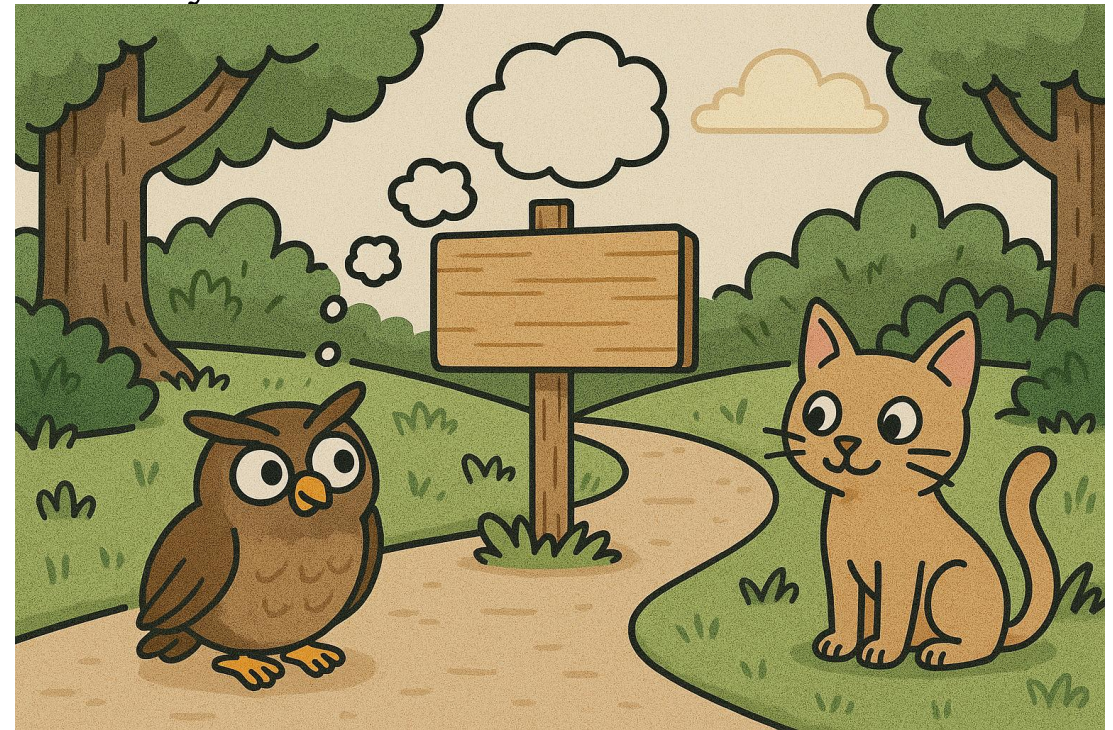


➤ Abstract Classes in Python

Python provides the abc module to create **abstract classes**.

📌 Rules:

- An abstract class must inherit from ABC (Abstract Base Class).
- It can have **abstract methods** (using @abstractmethod).
- You **cannot instantiate** an abstract class directly.



➤ Example of Abstract Class

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass
```

- ✓ Shape class is abstract.
- ✓ Any subclass must **implement** area() and perimeter().



Implementing Abstract Class

```
class Rectangle(Shape):
    def __init__(self, l, b):
        self.length = l
        self.breadth = b

    def area(self):
        return self.length * self.breadth

    def perimeter(self):
        return 2 * (self.length + self.breadth)

r = Rectangle(10, 5)
print("Area:", r.area())          # 50
print("Perimeter:", r.perimeter()) # 30
```

 If you don't implement all abstract methods, it will throw an error.



Attempting to Instantiate Abstract Class

```
shape = Shape() #  TypeError: Can't instantiate abstract class
```

Abstract classes are meant to be **inherited**, not used directly.



➤ Partial Abstraction

An abstract class can:

- Contain both **abstract** and **concrete methods**.
- Allow **default behavior** through implemented methods.

```
class Vehicle(ABC):  
    @abstractmethod  
    def start(self):  
        pass  
  
    def fuel_type(self):  
        print("Diesel or Petrol")  
  
class Car(Vehicle):  
    def start(self):  
        print("Car starts with key")  
  
c = Car()  
c.start()      # Car starts with key  
c.fuel_type()  # Diesel or Petrol
```



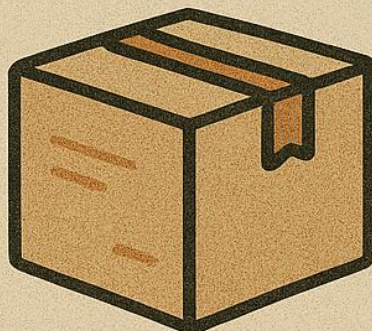
Why Use Abstraction?



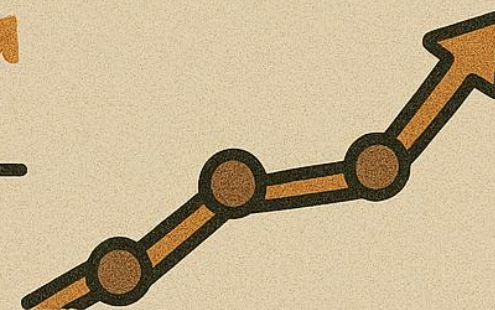
Hides internal logic



Only necessary details
are exposed



Enforce design
contracts



Easier to update
implementations

